

Bloodrose — Drift Detection Harness

Governed Compliance Infrastructure — v0.1

Project	Bloodrose
Artifact	Drift Detection Harness
Version	v0.1
Classification	Governed Compliance Infrastructure
Date	April 21, 2026
Status	Active
Depends On	Canonical Spec v0.1, Canonical Event Table v0.1, Modular Architecture Document v0.1, Operator Surface Spec v0.1

2. Purpose and Rationale

Drift is any deviation from canonical documents that was not pre-approved through the amendment process. It can be intentional — an unauthorized change introduced by a contributor who bypasses governance — or accidental — a misinterpretation of the spec, a copy-paste error, an incremental corruption introduced during development. In either case, the result is the same: the running system no longer reflects its canonical truth. The system says one thing; the documents say another. Drift has begun.

Drift is dangerous precisely because it is often invisible at the point of introduction. A single wrong constant — corruption delta set to 12 instead of 10. An event without an EVT-ID. A state mutation that bypasses the State Manager and writes directly to the game state object. Each of these seems small. Each is survivable in isolation. But drift is cumulative. Left unchecked, small deviations compound into systemic

divergence. The canonical documents become historical artifacts rather than living contracts. The system becomes ungoverned. In a project like Bloodrose — where corruption and redemption values drive narrative outcomes, where event priority determines player experience, where every threshold crossing triggers authored text — drift does not merely degrade quality. It destroys the authored experience.

The Drift Detection Harness is an automated verification system that continuously validates the running Bloodrose system against its canonical truth. It operates at three levels: **static analysis** (code and configuration inspection before runtime), **runtime monitoring** (live session observation during gameplay), and **post-session audit** (deep analysis of completed session logs). Together, these three layers form a comprehensive compliance mesh. No drift class escapes all three. Most are caught by at least two.

"Suggestion is not truth." The harness enforces this principle mechanically. If something is not in the canonical documents, it does not belong in the system. If it is in the canonical documents but implemented differently, the implementation is wrong — not the documents. The harness finds these discrepancies and flags them. It does not fix them. It does not modify state. It does not intervene in the game loop. It observes, it compares, it reports. Remediation is a human responsibility. Detection is the harness's sole purpose, and it performs that purpose without compromise.

3. Drift Taxonomy

The following table defines every class of drift the harness is designed to detect. Each drift class has a unique code, a severity level, and a concrete example drawn from the Bloodrose system.

Drift Class	Code	Description	Severity	Example
Unauthorized Event	DFT-001	An event fires that has no matching EVT-ID in the Canonical Event Table.	CRITICAL	A <code>collect_power_orb</code> event fires but EVT-IDs only cover petals and thorns.
State Bounds	DFT-002	A state	CRITICAL	Corruption is

Drift Class	Code	Description	Severity	Example
Violation		variable exceeds its canonical bounds (corruption or redemption outside 0-100).		set to 105 due to unclamped addition.
Forbidden State Transition	DFT-003	A state change occurs that violates a canonical constraint (e.g., thorn contact increasing redemption, or thorn contact triggering a win state).	CRITICAL	Player is in thorn zone and redemption increases in the same tick.
Missing Event Registration	DFT-004	An event defined in the Canonical Event Table has no corresponding registration in the Event System at runtime.	WARNING	EVT-024 (Score Milestone) exists in the table but was never loaded into the event registry.
Phantom Entity	DFT-005	An entity exists at runtime that has no corresponding schema in the Data Spec Layer.	CRITICAL	A "HealthPotion" entity appears in the entity list but no such type is defined canonically.
Hardcoded Constant	DFT-006	A game logic constant (corruption delta, threshold value, etc.) is hardcoded in a module	WARNING	Rules Engine checks if corruption $\geq$ 50 instead of reading the threshold from Data Spec.

Drift Class	Code	Description	Severity	Example
		instead of sourced from the Data Spec Layer.		
Direct State Mutation	DFT-007	A module modifies state by directly changing a property instead of using the State Manager's setter API.	CRITICAL	<code>gameState.corruption += 15</code> instead of <code>stateManager.setState('corruption', current + 15, 'eventSystem')</code>
Narrative Mismatch	DFT-008	The narrative text dispatched by an event does not match the canonical text defined in the Event Table.	WARNING	EVT-001 should output "A memory of betrayal burns through you..." but the implementation outputs a different string.
Priority Violation	DFT-009	Events are processed out of canonical priority order.	CRITICAL	A Priority 3 collection event is processed before a Priority 0 terminal event in the same tick.
One-Shot Repeat	DFT-010	A threshold event marked as non-repeatable fires more than once.	WARNING	EVT-020 (Corruption Warning at ≥ 50) fires a second time after corruption dips below 50 and rises again.
Undocumented State Variable	DFT-011	A state variable exists at runtime that is not	CRITICAL	A "mana" variable appears in the State Manager

Drift Class	Code	Description	Severity	Example
		defined in the Canonical Spec's state model.		but is not in the canonical state model.
Render State Mutation	DFT-012	The Render Layer modifies game state instead of being a pure consumer.	CRITICAL	The render module sets corruption based on a visual effect timer.
Aesthetic Violation	DFT-013	Visual or audio assets deviate from the canonical aesthetic direction (crimson, black, moonlit, gothic). <i>Note: this class requires manual review — automated detection is limited to palette analysis.</i>	INFO	A bright green particle effect is used for petal collection.
Interface Contract Violation	DFT-014	A module calls another module's API with incorrect parameter types or bypasses the defined interface.	CRITICAL	Event System directly accesses Entity Layer's internal array instead of using <code>getEntities()</code> API.
Amendment Bypass	DFT-015	A canonical document was modified without a corresponding entry in its Amendment Log.	CRITICAL	A new event EVT-040 appears in the Event Table but the AMD log has no entry for it.

4. Detection Layers

The harness operates across three detection layers, each targeting a different phase of the development and execution lifecycle. Together, they form a comprehensive compliance mesh.

4.1 Static Analysis Layer (Pre-Runtime)

Purpose: Catches drift before the code ever runs. Operates on source code, configuration files, and canonical documents.

Checks performed:

- **Event registry completeness:** Every EVT-ID in the Canonical Event Table must have a corresponding entry in the code's event registry. Every event in the code must have a matching EVT-ID. *Catches DFT-001, DFT-004.*
- **Constant sourcing:** Scan game logic modules for hardcoded numeric literals that match known canonical constants. Flag any that are not sourced from Data Spec. *Catches DFT-006.*
- **State variable inventory:** Compare runtime state variables against the canonical state model. Flag any additions or omissions. *Catches DFT-011.*
- **Entity type inventory:** Compare entity type definitions against canonical entity schemas. Flag unrecognized types. *Catches DFT-005.*
- **Interface compliance:** Verify that module-to-module calls match the defined API signatures in the Architecture Document. *Catches DFT-014.*
- **Narrative text comparison:** Diff all narrative output strings in code against the canonical text in the Event Table. *Catches DFT-008.*
- **Amendment log integrity:** Verify that any changes to canonical documents have corresponding Amendment Log entries. *Catches DFT-015.*

Tools: Linting scripts, custom AST analysis, text diffing, JSON schema validation.

When to run: On every code commit or pull request. Blocks merge if CRITICAL violations are found.

4.2 Runtime Monitoring Layer (During Session)

Purpose: Catches drift that only manifests during execution — logic errors, race conditions, state corruption.

Checks performed:

- **State bounds monitoring:** On every state mutation, verify the resulting value is within canonical bounds. *Catches DFT-002.*
- **Forbidden transition monitoring:** On every state mutation, verify it does not violate a canonical constraint (e.g., thorn contact never increases redemption). *Catches DFT-003.*
- **Event authorization:** On every event fire, verify the EVT-ID exists in the registered event table. *Catches DFT-001.*
- **Priority ordering:** On every tick where multiple events fire, verify they are processed in correct priority order. *Catches DFT-009.*
- **One-shot tracking:** On every threshold event fire, verify it has not fired before in this session. *Catches DFT-010.*
- **State mutation source:** On every state mutation, verify it came through the State Manager's setter API with a valid source identifier. *Catches DFT-007, DFT-012.*
- **Direct mutation detection:** Instrument state objects with proxies or `Object.freeze` in non-setter paths to catch unauthorized writes. *Catches DFT-007.*

Integration: Runtime checks feed into the Operator Surface's Canonical Compliance Panel. Violations trigger alerts per the Operator Surface alert hierarchy (INFO / WARNING / CRITICAL).

Performance Budget

Runtime checks must be lightweight — they run every tick. Target overhead: **less than 1ms per tick** for all runtime checks combined. If this budget is exceeded, the harness itself is in violation of its own governance constraints (see

4.3 Post-Session Audit Layer (After Session)

Purpose: Deep analysis of completed session data. Catches patterns that are invisible in real-time — statistical drift, cumulative balance issues, edge case sequences.

Checks performed:

- **Full session replay:** Replay the session log against a reference implementation and compare state history tick-by-tick. Any divergence is flagged. *Catches all drift classes.*
- **Event frequency analysis:** Compare event fire rates against expected ranges. Flag anomalies (e.g., an event that should fire 0–5 times per session firing 50 times). *Catches DFT-010 patterns.*
- **State trajectory analysis:** Plot corruption and redemption over time. Flag sessions where state behavior deviates from expected curves (e.g., corruption decreasing without any redemption-granting event). *Catches DFT-003 patterns.*
- **Coverage analysis:** Verify that all canonical events were reachable during the session. Flag events that could never fire given the session's entity placement. *Catches DFT-004 implications.*
- **Ending validation:** Verify that the ending type matches the terminal conditions (e.g., if `ending_type` is "mercy_dominant" but redemption was less than corruption at the time of altar contact, that is a drift). *Catches compound drift.*

Output: Post-session audit generates a Drift Report (see Section 7).

When to run: After every playtest session. Optionally automated in CI pipeline with synthetic sessions.

5. Assertion Library

The following assertions are the reusable validation primitives of the harness. Each assertion encapsulates a single verification rule and is referenced by the detection layers and canonical validation rules.

Assertion ID	Name	Signature	Description	Drift Classes Caught
ASSERT-001	assertStateBounds	assertStateBounds(key, value)	Verifies value is within canonical bounds for the given state key. For corruption and redemption, valid range is 0-100 inclusive.	DFT-002
ASSERT-002	assertEventRegistered	assertEventRegistered(evtId)	Verifies the event ID exists in the canonical event registry. Fails if the EVT-ID is unknown.	DFT-001, DFT-004
ASSERT-003	assertNoForbiddenTransition	assertNoForbiddenTransition(previousState, newState, source)	Verifies the state change does not violate a canonical constraint. For example, a source of "thorn" cannot produce an increase in redemption.	DFT-003
ASSERT-004	assertPriorityOrder	assertPriorityOrder(eventBatch)	Verifies events in a simultaneous batch are ordered by priority (Priority 0	DFT-009

Assertion ID	Name	Signature	Description	Drift Classes Caught
			first, highest numeric value last).	
ASSERT-005	assertOneShot Integrity	assertOneShot Integrity(evtId , firedSet)	Verifies a non-repeatable event has not already fired in the current session. firedSet is the set of EVT-IDs already triggered.	DFT-010
ASSERT-006	assertMutationSource	assertMutationSource(key, source, allowedSources)	Verifies the source of a state mutation is an authorized module within the allowedSources list. Render Layer is never an allowed source for game state mutations.	DFT-007, DFT-012
ASSERT-007	assertEntityTypeKnown	assertEntityTypeKnown(entityType)	Verifies the entity type exists in canonical schemas defined in the Data Spec Layer.	DFT-005
ASSERT-008	assertNarrativeMatch	assertNarrativeMatch(evtId, outputText, canonicalText)	Verifies narrative output matches canonical text. Match strictness is configurable (see Section 10).	DFT-008
ASSERT-009	assertStateVa	assertStateVa	Verifies the	DFT-011

Assertion ID	Name	Signature	Description	Drift Classes Caught
	riableKnown	riableKnown(key)	state variable key is defined in the canonical state model. Rejects any undocumented variables.	
ASSERT-010	assertEndingConditions	assertEndingConditions(endingType, state)	Verifies the ending type is justified by the terminal state values. Cross-references ending requirements from the Canonical Spec and Event Table.	Compound drift

6. Violation Response Protocol

When drift is detected, the response is determined by the violation's severity level. This protocol is non-negotiable — it applies uniformly to all contributors and all modules.

6.1 CRITICAL Violations

- **Immediate:** Alert dispatched to Operator Surface with CRITICAL tag. Event log entry created with full context (drift class, affected module, state snapshot, timestamp).
- **Runtime behavior:** In development builds, the game loop pauses and the violation is displayed on-screen. In production builds, the violation is logged silently (player experience is not interrupted) but the session is flagged as **non-canonical**.
- **Pipeline behavior:** In CI/CD, CRITICAL violations **block the build**. No merge is permitted until the violation is resolved.

- **Escalation:** CRITICAL violations must be resolved before the next playtest session.
- **Tracking:** Entered into the Drift Incident Log (see Section 8) with full context.

6.2 WARNING Violations

- **Immediate:** Alert dispatched to Operator Surface with WARNING tag.
- **Runtime behavior:** Logged but game continues normally in all build types.
- **Pipeline behavior:** In CI/CD, WARNING violations generate a report but **do not block the build**. However, three or more unresolved WARNINGS in the same module escalate to CRITICAL.
- **Tracking:** Entered into the Drift Incident Log. Must be resolved within two development cycles.

6.3 INFO Violations

- **Immediate:** Logged in Event Log. No alert dispatched.
- **Runtime behavior:** No impact on game execution.
- **Pipeline behavior:** Included in reports. No blocking.
- **Tracking:** Reviewed during periodic audits. May be reclassified to WARNING if a pattern emerges.

Escalation Rule

Three or more unresolved WARNING violations within a single module automatically escalate to CRITICAL severity. This prevents WARNING accumulation from becoming a vector for systemic drift.

7. Drift Report Format

The Drift Report is the primary output of the Post-Session Audit Layer. It is a structured JSON document containing a complete compliance assessment for a single session.

7.1 JSON Structure

Field	Type	Description
report_id	string	Unique identifier for this drift report (format: DR-YYYYMMDD-NNNN).
session_id	string	The session being audited. Matches the session ID from the Operator Surface.
timestamp	string (ISO 8601)	When the report was generated.
harness_version	string	Version of the Drift Detection Harness used to generate this report.
summary	object	Contains: total_violations (integer), critical_count (integer), warning_count (integer), info_count (integer), overall_status (enum: CLEAN, DRIFT_DETECTED, CRITICAL_DRIFT).
violations	array	Array of violation objects. Each contains: violation_id (string), drift_class (DFT-XXX), severity (CRITICAL / WARNING / INFO), timestamp_detected (ISO 8601), description (string), state_at_detection (state snapshot object), affected_module (string), resolution_status (enum: open, resolved,

Field	Type	Description
		accepted_risk), resolution_notes (string or null).
state_trajectory	array	Array of state snapshots at key moments: session start, each threshold crossing, and session end. Each snapshot includes tick, corruption, redemption, position, and active_events.
event_log_summary	object	Contains: total_events_fired (integer), breakdown_by_category (object mapping category names to counts), unauthorized_events (array of unrecognized EVT-IDs).
coverage_report	array	List of canonical events with each entry containing: evt_id, event_name, status (enum: fired, unfired, unreachable).
recommendations	array	Array of human-readable remediation suggestions (strings). Each recommendation references a specific violation and proposes a corrective action.

7.2 Human-Readable Summary

In addition to the JSON report, the harness generates a human-readable summary for inclusion in the Operator Surface's Session Summary panel. The summary follows this format:

Example: Human-Readable Drift Summary

Drift Report DR-20260421-0001 — Session SES-0042

Generated: 2026-04-21T17:00:00Z | Harness v0.1

Status: DRIFT_DETECTED

Total Violations: 3 (Critical: 1, Warning: 2, Info: 0)

Critical: DFT-007 — Direct state mutation in Rules Engine module.

gameState.corruption was modified directly at tick 1,247.

Warning: DFT-006 — Hardcoded constant in Rules Engine: threshold value 50 not sourced from Data Spec.

Warning: DFT-008 — Narrative text mismatch for EVT-001. Expected canonical string, received modified text.

Recommendation: Replace direct state assignment in Rules Engine with `stateManager.setState()` call. Refactor hardcoded threshold to reference Data Spec Layer. Correct narrative string for EVT-001.

8. Drift Incident Log

The Drift Incident Log is a persistent, append-only record of all drift incidents detected across sessions. Resolved incidents are never deleted — they are marked resolved with notes. This creates an institutional memory of governance challenges and their resolutions.

Incident ID	Date	Drift Class	Severity	Description	Affected Module	Session ID	Resolution Status	Resolution Date	Resolution Notes	Resolved By
DI-001	2026-04-20	DFT-001	CRITICAL	Unauthorized event collector_power_o	Event System	SES-0038	Resolved	2026-04-21	Remnant test code from entity	System Architect

Incident ID	Date	Drift Class	Severity	Description	Affected Module	Session ID	Resolution Status	Resolution Date	Resolution Notes	Resolved By
				rb detected during playtest. Event fired at tick 832 with no matching EVT-ID in the Canonical Event Table.					prototype. Event handler removed and pre-commit check added to prevent recurrence.	
DI-002	2026-04-21	DFT-006	WARNING	Hard coded constant detected during static analysis. Rules Engine contains if (corruptio	Rules Engine	N/A (Static)	Open	—	Scheduled for refactoring in next development cycle.	—

Incident ID	Date	Drift Class	Severity	Description	Affected Module	Session ID	Resolution Status	Resolution Date	Resolution Notes	Resolved By
				n >= 50) instead of referencing dataSpec.thresholds.corruption_warning.						

Log Governance

The Drift Incident Log is a living document. It is append-only. Resolved incidents are not deleted — they are marked resolved with notes. This creates an institutional memory of governance challenges and ensures patterns of recurring drift can be identified and addressed systemically.

9. Integration Architecture

The Drift Detection Harness integrates with the Bloodrose system at four connection points. At no point does the harness modify game state or interfere with the game loop. It is a pure observer.

9.1 Static Analysis Integration

- **Trigger:** Runs as a pre-commit hook or CI pipeline stage.
- **Reads:** Source code, Data Spec Layer configuration, Canonical Event Table (as JSON or structured data), entity schemas, canonical documents.
- **Outputs:** Pass/fail status, violation report (JSON).

- **Blocking:** CRITICAL violations block merge. WARNING violations generate reports but do not block.

9.2 Runtime Integration

- **Architecture:** The harness runs as a lightweight module **alongside** the game loop — not inside it. It observes; it never modifies.
- **Reads:** State Manager mutations (via subscriber API), Event System dispatches (via event log), Entity Layer state.
- **Feeds:** Operator Surface Canonical Compliance Panel.
- **Performance budget:** Less than 1ms per tick total for all runtime checks combined.

9.3 Post-Session Integration

- **Trigger:** Automatically on session end, or manually by operator via the Operator Surface.
- **Reads:** Full session event log (exported from Operator Surface), State Manager snapshots.
- **Outputs:** Drift Report (JSON), human-readable summary.
- **CI Integration:** Can be integrated into CI pipeline with synthetic playtest sessions for regression drift detection.

9.4 Operator Surface Integration

- The Operator Surface's **Canonical Compliance Panel** is the primary real-time display for harness output.
- **Drift indicator** (GREEN / AMBER / RED) is driven by harness results:
 - **GREEN:** No violations detected. System is canonical.
 - **AMBER:** WARNING or INFO violations detected. System is operational but drifting.
 - **RED:** CRITICAL violation detected. System integrity is compromised.

- Violation details are displayed in the Compliance Panel's violation log.
- Session summary includes the drift report summary (see Section 7.2).

10. Harness Configuration

The following parameters control harness behavior. All parameters are configurable via the Data Spec Layer in modular and production builds. In the prototype phase, they are set as constants in the harness module.

Parameter	Type	Default	Description
enabled_layers	array<string>	["static", "runtime", "post_session"]	Which detection layers are active. All three are enabled by default.
runtime_check_interval	string	"every_tick"	How often runtime checks execute. Options: <code>every_tick</code> , <code>every_nth_tick</code> , <code>on_state_change_only</code> .
severity_escalation_threshold	integer	3	Number of unresolved WARNINGS within a single module before escalation to CRITICAL.
pause_on_critical	boolean	true (dev) / false (prod)	Whether to pause the game loop on CRITICAL violation. Must be false in production to preserve player experience.
narrative_match_strictness	string	"exact"	How strictly narrative text is compared. Options: <code>exact</code> (byte-for-byte), <code>normalized_whitespace</code> (ignores whitespace)

Parameter	Type	Default	Description
			differences), fuzzy_90percent (90% similarity threshold).
state_snapshot_interval	integer	60	Seconds between automatic state snapshots for post-session analysis.
export_format	string	"json"	Drift report output format. Options: json, csv, html.
log_retention	string	"all"	How long drift incident logs are retained. Options: all (indefinite), 30_days, 90_days.

11. Canonical Validation Rules

The following table is the comprehensive reference of every canonical rule the harness enforces. Each rule traces back to a specific source document and section, maps to an assertion primitive, and identifies the drift class it guards against.

Rule ID	Rule Description	Source Document	Section Ref	Assertion Used	Drift Class
CVR-001	Corruption must remain within 0-100 inclusive.	Canonical Spec v0.1	Section 7	ASSERT-001	DFT-002
CVR-002	Redemption must remain within 0-100 inclusive.	Canonical Spec v0.1	Section 7	ASSERT-001	DFT-002
CVR-003	Thorn contact	Canonical Spec v0.1	Section 9	ASSERT-003	DFT-003

Rule ID	Rule Description	Source Document	Section Ref	Assertion Used	Drift Class
	must not increase redemption .				
CVR-004	Thorn contact must not trigger a win state.	Canonical Spec v0.1	Section 9	ASSERT-003	DFT-003
CVR-005	Every fired event must have a registered EVT-ID in the Canonical Event Table.	Canonical Event Table v0.1	Section 4	ASSERT-002	DFT-001
CVR-006	Terminal events (Priority 0) override all other events in the same tick.	Canonical Event Table v0.1	Section 4	ASSERT-004	DFT-009
CVR-007	Threshold events fire only on first crossing. Non-repeatable events must not fire more than once per session.	Canonical Event Table v0.1	Section 4	ASSERT-005	DFT-010
CVR-008	Collection events consume the collectible instance. The entity	Canonical Event Table v0.1	Section 4	ASSERT-007	DFT-005

Rule ID	Rule Description	Source Document	Section Ref	Assertion Used	Drift Class
	must be removed from the active entity list after collection.				
CVR-009	All state effects are deterministic. No random modifiers are permitted in state mutations.	Canonical Event Table v0.1	Section 4	ASSERT-006	DFT-007
CVR-010	Render Layer must not modify game state. It is a pure consumer of state data.	Modular Architecture v0.1	Section 4.7	ASSERT-006	DFT-012
CVR-011	Win condition requires redemption ≥ 65 and altar reached.	Canonical Spec v0.1	Section 10	ASSERT-010	Compound
CVR-012	Mercy Dominant ending requires redemption $>$ corruption at time of altar contact.	Canonical Event Table v0.1	EVT-031	ASSERT-010	Compound
CVR-013	Bound but Unbroken	Canonical Event Table	EVT-032	ASSERT-010	Compound

Rule ID	Rule Description	Source Document	Section Ref	Assertion Used	Drift Class
	ending requires redemption $\leq$ corruption at time of altar contact.	v0.1			
CVR-014	Altar rejected if redemption < 65 . Player cannot win with insufficient redemption .	Canonical Event Table v0.1	EVT-033	ASSERT-010	Compound
CVR-015	All state mutations must go through the State Manager's setter API. No direct property assignment is permitted.	Modular Architecture v0.1	Section 4.2	ASSERT-006	DFT-007

12. Implementation Phases

12.1 Phase 1 — Prototype (v0.1)

Minimal assertion set embedded directly in the single-file prototype. This phase establishes the compliance habit without requiring infrastructure.

- `console.assert()` or `console.warn()` for state bounds and forbidden transitions.
- Manual event ID verification — no automated registry check yet.

- No formal Drift Report — violations are visible in the browser console only.
- Core assertions active: **ASSERT-001** (state bounds), **ASSERT-003** (forbidden transitions), **ASSERT-006** (mutation source).
- The harness is embedded in the main game file, not a separate module.

12.2 Phase 2 — Modular

Full assertion library implemented as a standalone harness module within the 8-module architecture.

- All 10 assertions active and independently testable.
- Static analysis scripts for pre-commit checks (event registry, constant sourcing, state variable inventory).
- Runtime monitoring integrated with the Operator Surface's Canonical Compliance Panel.
- Post-session audit with JSON Drift Report generation.
- Drift Incident Log maintained as a living document.
- Configurable harness parameters via Data Spec Layer.

12.3 Phase 3 — Production Pipeline

Full CI/CD integration with automated enforcement and historical analysis.

- CI/CD pipeline integration — static analysis gates merges automatically.
- Automated synthetic playtest sessions for regression drift detection.
- Historical drift trend analysis across sessions (drift frequency, time-to-resolution, recurring modules).
- Harness configuration management via Data Spec Layer with environment-specific overrides (dev/staging/prod).
- Role-based access to drift reports and incident logs.
- Alerting integration — CRITICAL violations can trigger external notifications (email, Slack, webhook).

13. Governance Constraints for the Harness Itself

The Harness Is Governed

The Drift Detection Harness is itself a governed artifact. It is subject to the same amendment process as every other canonical document in the Bloodrose project. The following constraints are non-negotiable.

1. **Observation only.** The harness must never modify game state. It observes, compares, and reports. If a harness module is found to have mutated state, that is a CRITICAL governance violation of the harness itself.
2. **Governed validation rules.** The Canonical Validation Rules table (Section 11) is a governed document subject to the amendment process. Adding, removing, or modifying a CVR entry requires a versioned amendment to this document.
3. **Governed drift classes.** New drift classes (DFT-XXX) or assertions (ASSERT-XXX) require a versioned amendment to this document. Ad-hoc detection rules are forbidden.
4. **Performance budget.** The harness must not introduce performance degradation exceeding its stated budget: less than 1ms per tick for the runtime layer. If the budget is exceeded, the harness configuration must be adjusted (e.g., switching to `on_state_change_only` mode) — not the budget.
5. **No false positive suppression.** False positives must be investigated and either corrected (harness bug) or reclassified (new drift class or rule adjustment). Silencing alerts without investigation is **forbidden**.
6. **Mandatory in production.** The harness is not optional in production builds. Runtime monitoring may be configured for performance (e.g., `on_state_change_only` instead of `every_tick`) but must not be disabled entirely. The `enabled_layers` parameter must always include `"runtime"`.

14. Amendment Log

Amendment ID	Date	Description	Author	Approved By
AMD-000	April 21, 2026	Initial Drift Detection Harness created. Establishes drift taxonomy (15 classes), assertion library (10 assertions), three detection layers, violation response protocol, drift report format, canonical validation rules (15 rules), and three-phase implementation plan.	System Architect	Pending Review

Bloodrose — Drift Detection Harness v0.1 — Governed Compliance Infrastructure
Drift is silent, cumulative, and corrosive. This harness catches it before it compounds.